

House Price Prediction from Specs and Exterior Photos

CSU Bakersfield, CMPS 4450 Data Mining Final

Noah Gallego

The question

specs already get us most of the way. can the photo close the gap?

what we know about each house

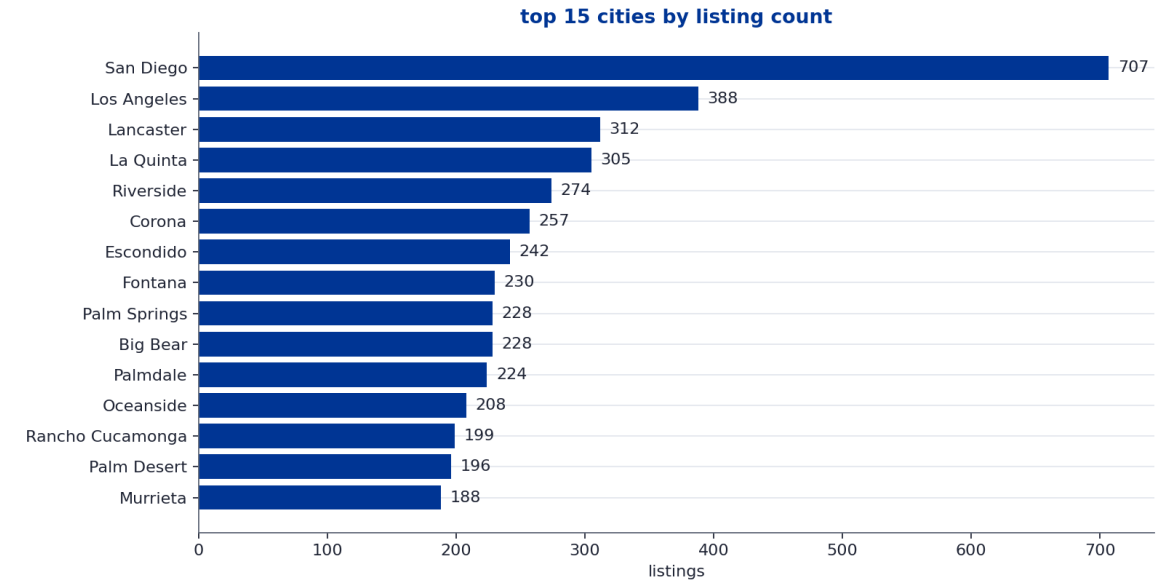
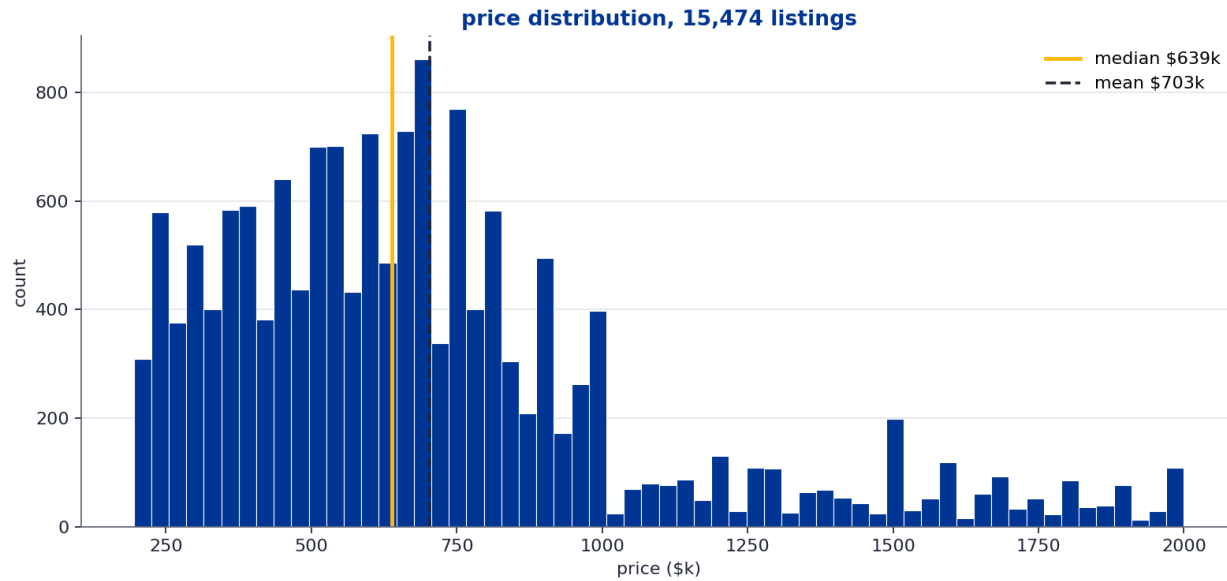
beds, baths, square footage, city, and one exterior photo.

the experiment

fit a model on specs alone. then add features from the photo.
does the second model beat the first?

Data

15,474 southern california listings, \$195k to \$2M

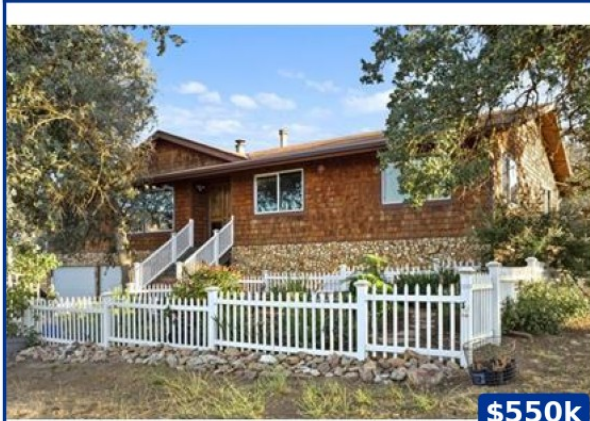


What a row looks like

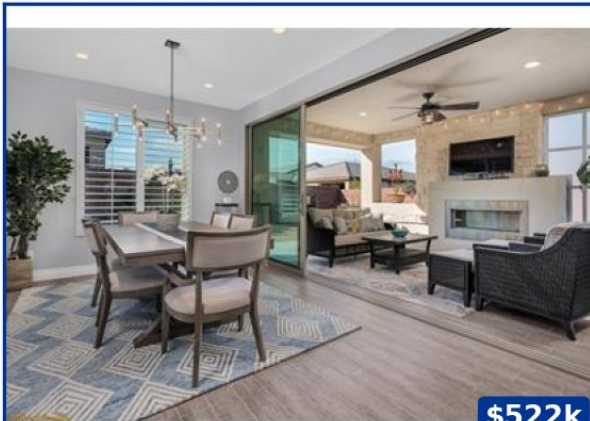
one photo plus a few numeric fields per house

six houses, drawn at random

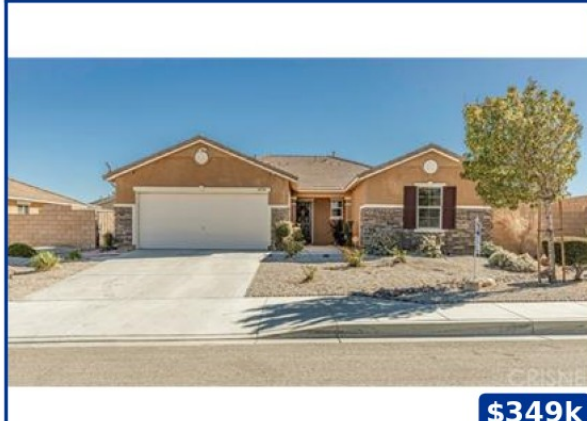
Caliente
3bd / 2.1ba / 2,040sf



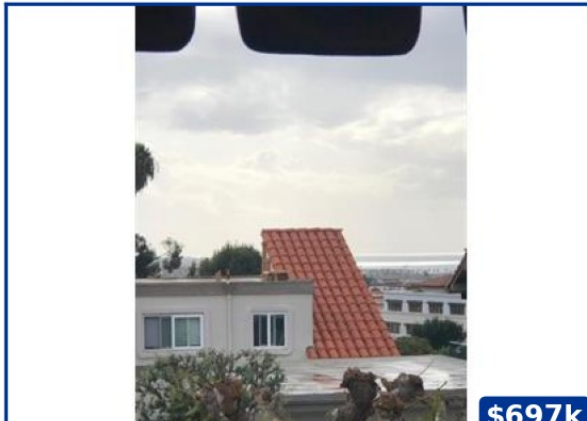
Indio
2bd / 2ba / 1,909sf



Lancaster
3bd / 2ba / 1,504sf



San Clemente
3bd / 2.1ba / 1,270sf



Apple Valley
4bd / 2.1ba / 3,157sf



Banning
3bd / 2ba / 1,122sf



Train, validation, test

how we use the 15,474 listings

how we set it up

shuffle the listings and split 65 / 15 / 20 into train, validation, and test
fit the models on train
pick the model and its settings (tree depth, KNN k) by validation error
re-run the whole comparison on 500 random reshuffles to check stability
open the test set once, at the end, with every choice already locked

why this design

one fixed split swings by several thousand dollars just from luck.
the reshuffles smooth that out, and the test number stays honest because
it never gets to inform a choice.

Eight columns from Kaggle

what arrived in the CSV, and what we do with each one

column	example	action
image_id	0	drop
street	1317 Van Buren Avenue	drop
citi	Salton City, CA	convert to coordinates
n_citi	317	drop
bed	3	keep
bath	2.0	keep
sqft	1,560	keep
price	\$201,900	target

six features go into every model: bed, bath, sqft, city_lat, city_lon, region_id.

City names become coordinates

we look each city up once with a geocoding API

the lookup

city	latitude	longitude
Beverly Hills, CA	34.0736	-118.4004
San Diego, CA	32.7157	-117.1611
Bakersfield, CA	35.3733	-119.0187
Salton City, CA	33.3030	-115.9486

why this works

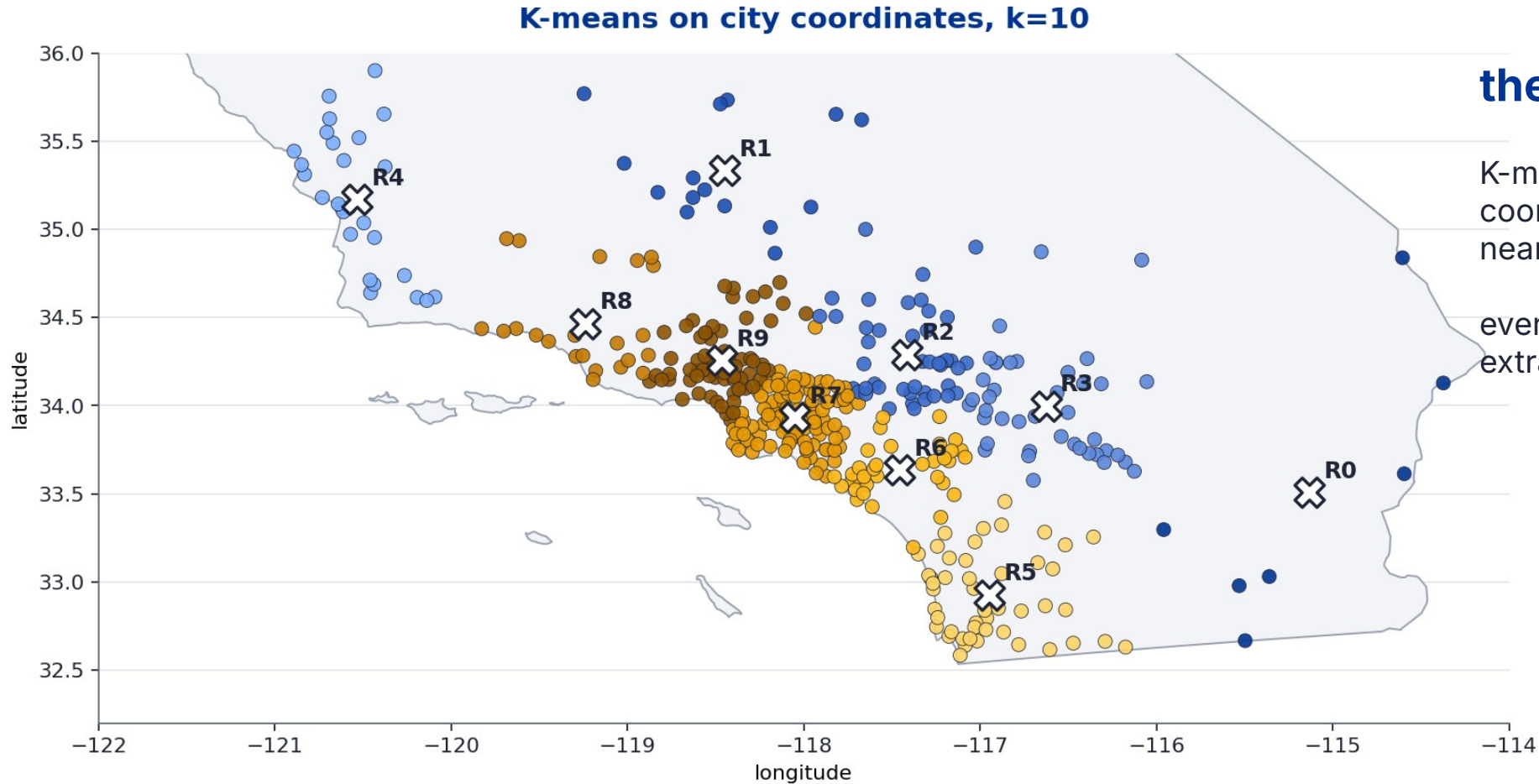
415 unique cities ⇒ 415 API calls, cached once on disk.

every row gets its city's latitude and longitude.

now 'nearest neighbor' actually means geographically near, not just same city name.

Grouping cities into regions

K-means on the 415 city coordinates, $k=10$



the idea

K-means takes the coordinates and groups nearby cities together.

every row gets one extra feature: region_id.

Normalize so no column dominates

rescale every feature to the same range

feature	raw range	after min-max
bed	1 to 6	0 to 1
bath	1 to 5	0 to 1
sqft	500 to 5,400	0 to 1
city_lat	32.5 to 35.4	0 to 1
city_lon	-119.5 to -114.5	0 to 1
region_id	0 to 9	0 to 1

min-max: $(\text{value} - \text{min}) / (\text{max} - \text{min})$

without normalizing, sqft (range 4,900) drowns out bed (range 5).
every column is now between 0 and 1, so they all matter the same to KNN's distance.

Two models

to establish a text-only baseline before we touch the photos

multiple linear regression

one weight per feature, all summed plus an intercept.

prediction =
 $w_0 + w_1 \cdot \text{bed} + w_2 \cdot \text{bath} + w_3 \cdot \text{sqft}$
 $+ w_4 \cdot \text{city_lat} + w_5 \cdot \text{city_lon} + w_6 \cdot \text{region}.$

fast and transparent. assumes the relationship is a straight line.

KNN regressor

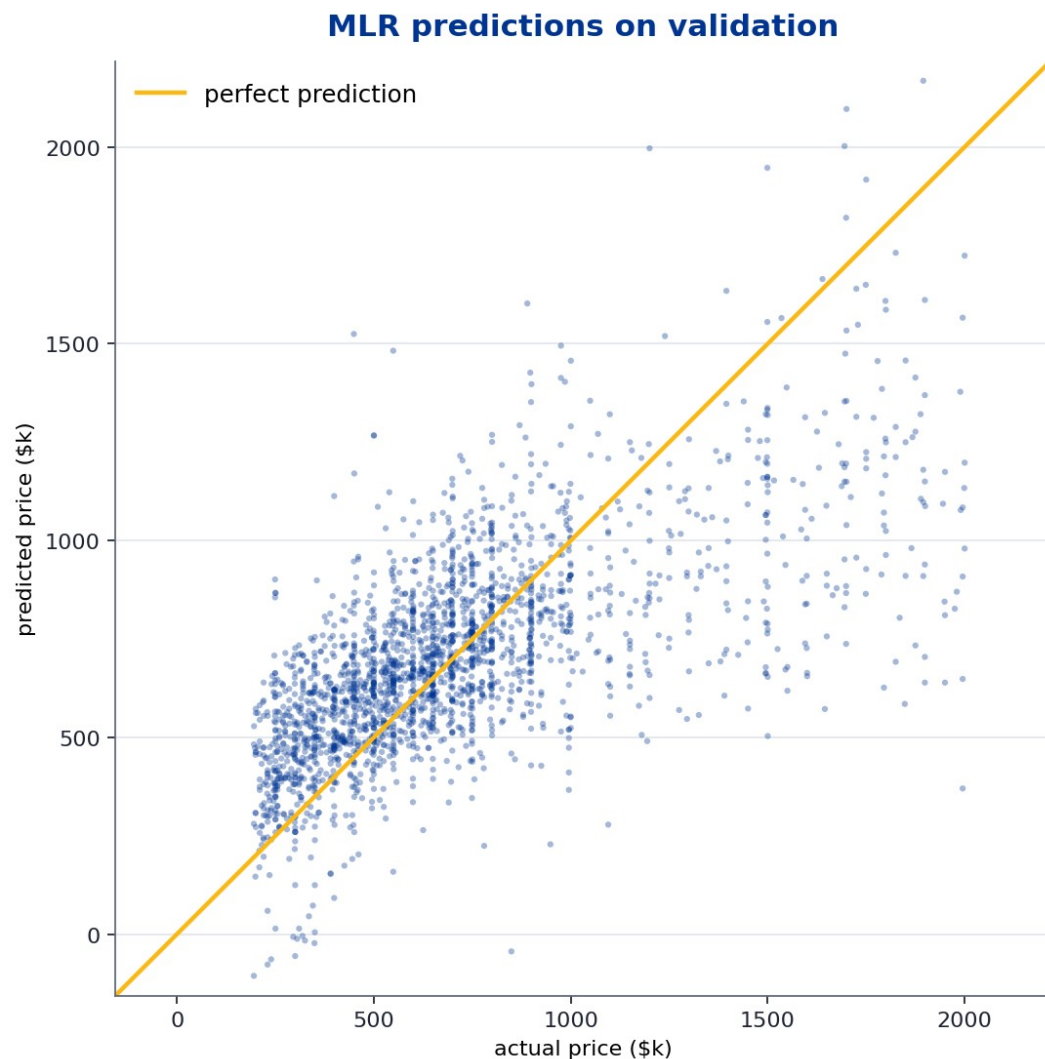
for a new house, find the k closest houses in feature space.

predict by averaging their prices (regression, not voting).

needs normalized features. tuned by k and by the distance function.

Multiple linear regression

validation performance, predicted vs actual price



results

MAE	\$200k
RMSE	\$280k
median error	\$150k
max error	\$1,625k
SSE	\$181.51T
within \$50k	18.7%
within \$100k	35.8%
within \$200k	63.0%

KNN, four distance functions

each box summarizes 200 random reshuffles at that k

reading the plot

each panel is a different distance function.

each box: spread of validation MAE at that k.

gold line connects the medians.

lower is better.

Where the baseline misses

the top six errors split into two kinds

at the \$1,995k dataset cap — 38 listings share this exact price

Tehachapi, 4bd/1.1ba/1,980sf



Morro Bay, 6bd/3ba/1,838sf



San Marcos, 3bd/3ba/3,222sf



worst misses below the cap — real model failures

Oxnard, 2bd/2ba/941sf



San Jacinto, 5bd/4ba/2,392sf



Shadow Hills, 3bd/2ba/2,685sf



Three things the photo could fix

the case for adding image features

condition

same specs, different
upkeep and landscaping.
that lives in the photo.

size beyond sqft

story count, lot size,
yard space. specs miss
those. the photo doesn't.

neighborhood look

trees, driveway, fencing,
lighting. signals the
street, not just the city.

Why not just feed in the pixels?

the dimensionality wall

each photo is $128 \times 128 \times 3 = 49,152$ pixel values

training set has roughly 10,000 houses

more inputs than examples

the model can fit anything in training and learn nothing useful.

linear regression is underdetermined.

trees and KNN see pixel positions as unrelated columns — shift the house ten pixels left and every input value changes.

A filter is a 3×3 window that slides across the photo

the kernel is fixed, the output is a smaller summary

photo, sobel-y kernel sliding



filter response, building up



at each position, multiply the 9 pixels under the window by the kernel weights and sum.
that sum becomes one output pixel. here the kernel detects horizontal edges (sobel-y).

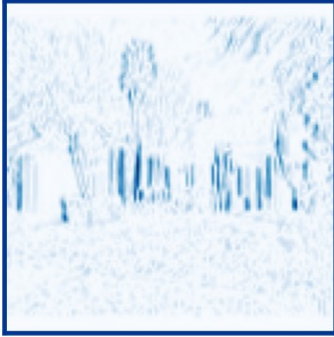
Our six filters

one photo, six response maps

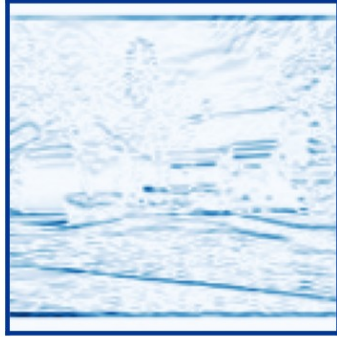
original



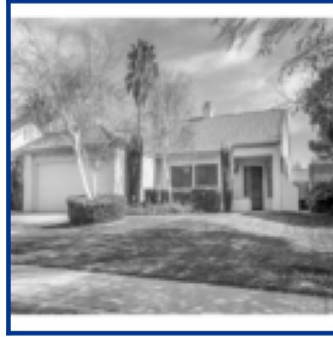
sobel-x
vertical edges



sobel-y
horizontal edges



brightness
light vs dark



blue dominance
sky / pool



green dominance
vegetation



red dominance
roof / brick



edges catch the roofline and window frames. color dominances pick out sky, lawn, and roof.

Pooling: 128×128 down to 4×4

each filter response collapses to 16 numbers

the idea

split each 128×128 response into a 4×4 grid of 32×32 patches.

take the mean of each patch.

now each filter is 16 numbers instead of 16,384.

what the model sees

six filters x 16 cells = 96 spatially-aware numbers per photo.

plus the 4 specs. 100 features total per house.

What the model sees, end to end

100 numbers per house

from the listing

beds
baths
square footage
typical price in that city

from the photo

6 filters, each pooled to 4×4
= 96 numbers describing
what's where in the photo

Did spatial features help?

validation MAE, by feature set and model

feature set	linear	tree	knn	best
specs only	\$165k	\$144k	\$150k	\$144k
specs + spatial (96)	\$164k	\$156k	\$218k	\$156k
spatial only	\$269k	\$284k	\$265k	\$265k

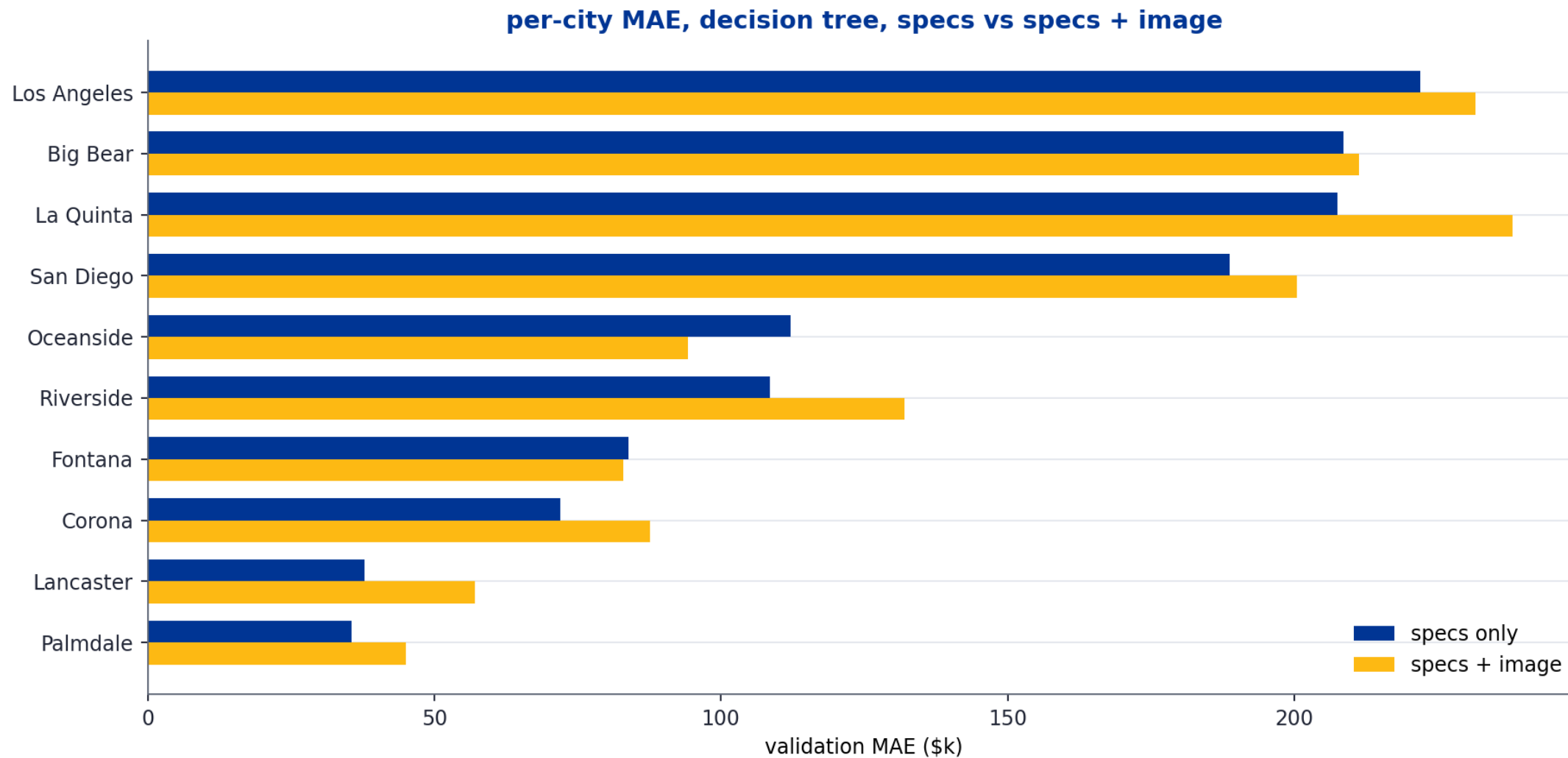
spatial features alone get to \$284k — barely better than guessing the average.

adding them on top of specs makes tree and KNN worse, not better.

specs still win.

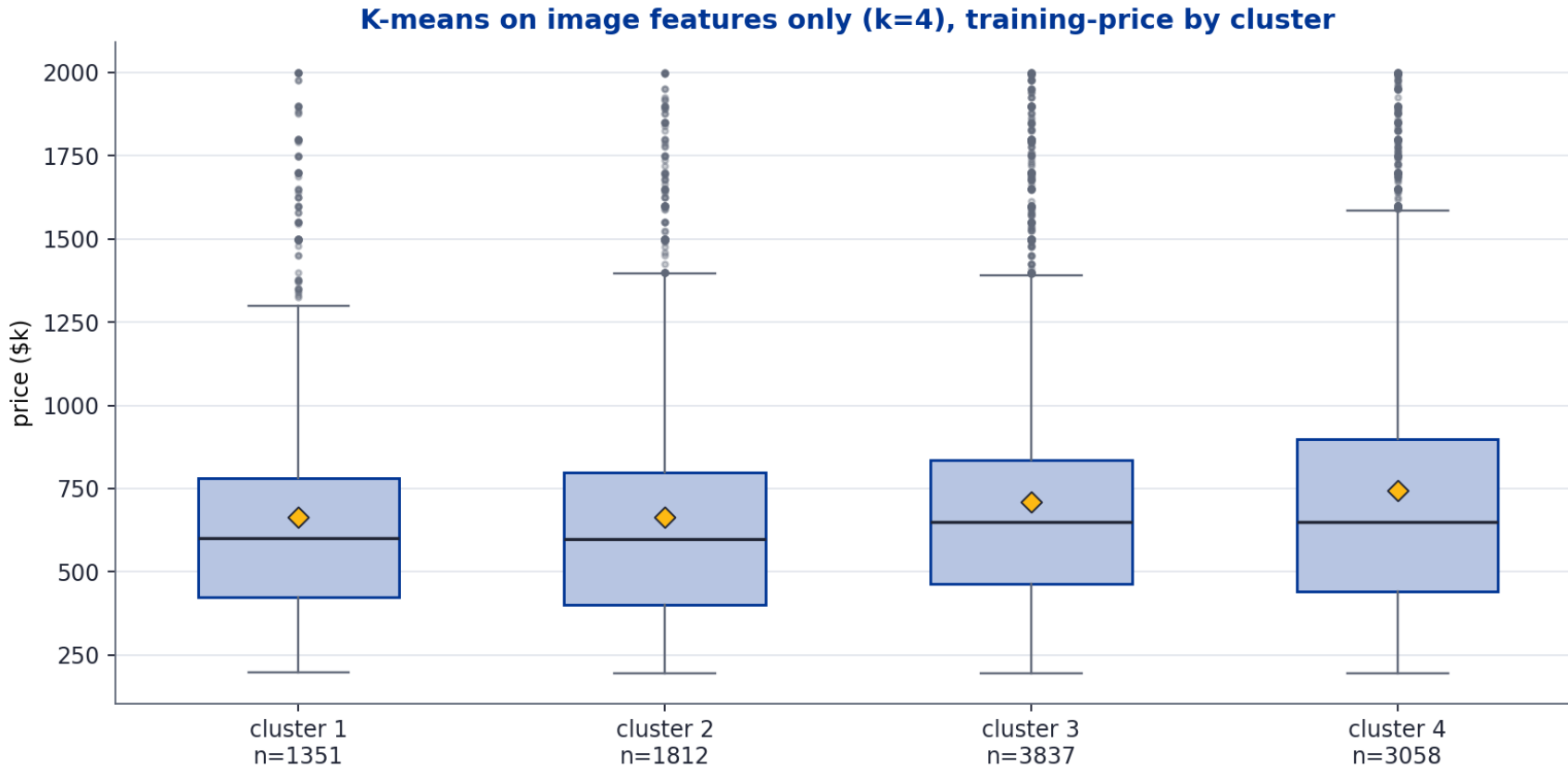
Per-city, with and without the photo

two cities benefit a little. the rest get worse.



K-means on photo features alone

if photos carried price signal, clusters would land at different prices



what we found

four clusters, mean prices land within \$80k of each other.

the photo features don't separate price.

Final test results

trained on train + val, evaluated once on the 3,095 held-out test rows

model	features	MAE	RMSE	<\$50k	<\$100k
constant	none	\$270k	\$366k	13.9%	27.1%
linear regression	specs	\$164k	\$233k	23.2%	44.5%
linear regression	specs + spatial	\$163k	\$233k	23.1%	44.8%
decision tree, d=15	specs	\$142k	\$218k	32.9%	54.3%
decision tree, d=10	specs + spatial	\$154k	\$234k	29.6%	50.9%
KNN, k=15	specs	\$150k	\$224k	29.4%	51.0%
KNN, k=15	specs + spatial	\$208k	\$291k	15.8%	33.9%

specs-only decision tree wins on test.

MAE \$142k. 33% of guesses within \$50k, 54% within \$100k.

Conclusion

specs already carry the price signal.
the photo, hand-summarized in numpy, does not add anything on top.

what we learned

feeding raw pixels doesn't work — too many inputs, too few examples,
and classical models can't see that adjacent pixels are related.

we built the principled fix: six hand-designed filters plus 4×4 pooling.
the model gets spatial info — sky on top, lawn on bottom, edges where the roof is.
this still doesn't beat specs.

the signal exists inside the photo, but recovering it needs filters that LEARN
what to look for. that's what CNNs do. hand-designed kernels can only go so far.